

Automated Data Cleaning

A stored procedure that cleans the US household income data and reruns itself on a schedule, instead of being cleaned by hand each time the source table refreshes.

Tools SQL. Stored procedures, scheduled events, window functions

Data Same source as the US Household Income project: 32,526 areas across 50 states, DC and Puerto Rico

The problem

In the earlier household income project, the cleaning was a one-off: a set of queries run once by hand against a static file. That works for a single analysis, but it breaks the moment the source table is refreshed with a new export, since every fix has to be reapplied manually. This project turns the same cleaning steps into something that runs on its own.

What I built

A stored procedure, `Copy_And_Clean_Data`, that takes the raw table and produces a cleaned copy of it from scratch every time it runs:

- Creates the cleaned table if it does not already exist, with a Timestamp column added to mark when each copy was produced
- Copies every row from the raw table into it
- Removes duplicate rows using `ROW_NUMBER()`, partitioned by ID and timestamp so only the current run's duplicates are considered
- Fixes the known data issues: the "georia" typo, "Boroughs" merged into "Borough", "CPD" merged into "CDP"
- Standardises County, City, Place and State_Name to uppercase, so the same place typed in different cases is never treated as two different places

Putting the cleaning inside a procedure means it is called the same way every time, rather than retyped or copy-pasted, which is where manual cleaning usually drifts.

Running it on a schedule

A scheduled event calls the procedure automatically every 30 days, so the cleaned table refreshes on its own without anyone needing to remember to run it.

Piece	What it does
<code>Copy_And_Clean_Data</code>	The stored procedure. All of the cleaning logic lives here, written once
<code>run_data_cleaning</code>	A scheduled event that calls the procedure automatically every 30 days

Checking it worked

After calling the procedure, three checks confirm the output is correct: a `ROW_NUMBER()` query to confirm no duplicate IDs remain, a row count against the expected total, and a count of areas by state to spot anything that merged incorrectly during the uppercase standardisation.

Why this matters

The underlying problem is the same one real pipelines run into: source data doesn't arrive clean once, it arrives messy repeatedly. A procedure plus a schedule is the simplest version of the fix, and it is the same pattern a scheduled job or an orchestration tool would wrap around a larger pipeline. The decision this supports is trust: whoever queries `USHouseholdIncome_Cleaned` can rely on it being current and deduplicated without checking it by hand first.

Limitations

The 30-day schedule is fixed rather than triggered by the source table actually changing, so a refresh that lands between runs waits for the next scheduled pass. The procedure also recreates the full cleaned table from the raw data each time rather than processing only new rows, which is the simpler approach and the right one at this size, but would need to change if the source table grew much larger.

Portfolio project built on the same public US household income dataset used in the US Household Income project.